

Introdução

O que é o Ecossistema AIDD?

O Ecossistema AIDD é um conjunto de ferramentas que transformam uma ideia em software testado e pronto para produção. Funciona como uma **fábrica de software**: cada ferramenta é uma estação de trabalho especializada, e todas seguem as mesmas regras de qualidade.

O que significa AIDD? AI-Driven Development, ou "Desenvolvimento Impulsionado por Inteligência Artificial". Na prática, significa que as ferramentas usam IA para acelerar o trabalho, mas sempre com verificações automáticas que garantem que o resultado está correto.

Para quem é este manual?

- Desenvolvedores que querem criar software mais rápido e com menos erros
- Empreendedores que têm uma ideia de app e querem ver ela funcionando
- Equipes que querem padronizar como criam e mantêm projetos
- Qualquer pessoa que queira entender como funciona uma fábrica de software moderna

Como usar este manual

O manual é dividido em duas partes:

1. **Manual de Referência** — Explica cada ferramenta, o que faz e como usar
2. **Caso Prático Completo** — Mostra tudo funcionando do zero até produção, construindo um sistema de clínica médica

Se você é iniciante, comece pelo Caso Prático. Se já conhece o ecossistema, use a referência rápida.

Parte 1 — Manual de Referência

Conceitos Básicos

Antes de entrar nas ferramentas, vamos explicar alguns termos que vão aparecer:

Terminal / Linha de Comando: É um programa onde você digita comandos em vez de clicar botões. Parece uma tela preta com letras. No Windows, é o "Prompt de Comando" ou "PowerShell". No Mac/Linux, é o "Terminal". É onde você vai digitar os comandos do ecossistema.

CLI: Command Line Interface, ou "Interface de Linha de Comando". É a forma de usar o ecossistema pelo terminal, digitando comandos como `python ecossistema.py generate "minha ideia"`.

Slash Command: É um comando que você digita no chat de um assistente de IA (como o Claude Code ou o Cursor). Começa com barra: `/forge`, `/generate`, `/master`. É a forma mais fácil de usar o ecossistema.

Quality Gate: É uma verificação automática que o ecossistema faz antes de considerar algo "pronto". Funciona como um detector de mentiras: se o código não estiver correto, o gate bloqueia. São como os portões de um aeroporto — nada passa sem ser verificado.

Projeto Legado: Um projeto de software que já existe, geralmente criado há muito tempo e sem seguir padrões modernos.

Vendor Lock-in: Quando você fica "preso" a uma plataforma porque é difícil sair dela. Por exemplo, criar um app no Lovable e não conseguir colocar em outro servidor sem reescrever tudo.

Clean Architecture: Um estilo de organizar código que separa as partes em camadas (regras de negócio, banco de dados, interface). É como organizar uma casa: cozinha fica na cozinha, quarto no quarto — tudo no lugar certo.

Fatia Vertical (Vertical Slice): Uma parte completa do sistema que inclui tudo o que precisa: regras, banco de dados e interface. É como um módulo independente que funciona sozinho, mas se conecta com os outros.

Mock: Uma simulação falsa de algo que deveria ser real. Por exemplo, simular um banco de dados em vez de usar um de verdade. O ecossistema proíbe mocks — tudo precisa ser real.

Result Monad: Um padrão de programação onde toda operação retorna "deu certo" ou "deu errado", em vez de quebrar o programa. É como um semáforo: verde (sucesso) ou vermelho (falha), sem intermediários.

As 6 Ferramentas

O ecossistema tem 6 ferramentas. Cada uma resolve uma etapa diferente do processo de criar software.

AIDD Forge — O Instalador de Governança

O que é: Uma ferramenta que instala regras de qualidade em qualquer projeto.

Quando usar: Quando você tem um projeto existente (novo ou legado) e quer que ele passe a seguir padrões de qualidade automáticos.

Analogia: É como instalar um sistema de segurança num prédio — depois que instalado, toda obra futura é verificada automaticamente.

Como usar pelo terminal:

```
# Instalar governança no projeto atual
python ecossistema.py forge init .

# Injetar um componente específico (skill, regra, etc.)
python ecossistema.py forge inject skill nome-da-skill
```

Como usar no chat:

```
/forge .
```

O que acontece quando você roda:

1. O Forge copia os arquivos de governança para o seu projeto
2. Instala 7 verificações automáticas (gates) que validam o código
3. Cria ponteiros para que qualquer assistente de IA entenda as regras
4. Configura o Git para manter a consistência dos arquivos

AIDD Generator — A Fábrica de Software

O que é: Uma ferramenta que cria um projeto completo a partir de uma ideia escrita em linguagem normal.

Quando usar: Quando você tem uma ideia de software e quer ver ela funcionando sem escrever código manualmente.

Analogia: É como descrever o que você quer num guardanapo e receber o produto pronto — com banco de dados, testes e documentação.

Como usar pelo terminal:

```
# Criar projeto (fases 1-7: estrutura + docs)
python ecossistema.py generate "sistema de agendamento para clínica" --pasta ./cl

# Criar com código funcional (fase 8: gera código via IA)
python ecossistema.py generate "sistema de agendamento" --pasta ./clinica --imple
```

Como usar no chat:

```
/generate sistema de agendamento para clínica médica
```

O que acontece quando você roda:

O Generator executa 8 etapas (chamadas de "fases"):

Fase	Nome	O que faz	Tempo
1	Pesquisa	Coleta informações sobre o que você quer	Rápido
2	Análise	Decompe o projeto em partes menores	Rápido
3	Design	Decide a arquitetura (como as peças se encaixam)	Rápido
4	Planejamento	Define tecnologias e estrutura	Rápido
5	Criação	Gera schemas, scripts e artefatos	Rápido
6	Documentação	Cria documentação completa (HTML, Markdown, PDF)	Rápido
7	Auto-crítica	Verifica se tudo está correto	Rápido
8	Implementação	<i>(opcional)</i> Gera código funcional via IA	Lento

Importante: As fases 1-7 são determinísticas — sempre funcionam igual. A fase 8 usa IA e pode gerar código que precisa de ajustes (taxa de sucesso de 55% a 91%, dependendo da complexidade).

AIDD Master — O Construtor de Módulos

O que é: Uma ferramenta que adiciona novas partes (módulos) a um projeto que já existe.

Quando usar: Quando seu projeto já está funcionando e você quer adicionar uma funcionalidade nova.

Analogia: Se o Generator constrói o prédio inteiro, o Master constrói um cômodo novo — mantendo o mesmo padrão de fundação e encanamento do resto.

Como usar pelo terminal:

```
# Adicionar módulo de relatórios financeiros
python ecossistema.py master add-module relatorios-financeiros
```

Como usar no chat:

```
/master relatorios-financeiros
```

O que acontece quando você roda:

O Master cria uma "fatia vertical" completa:

```
src/modules/relatorios-financeiros/
├─ domain/           ← Regras de negócio (o que o módulo faz)
├─ application/      ← Casos de uso (quando e como usar)
├─ infrastructure/   ← Banco de dados (onde guardar informações)
├─ interfaces/       ← Rotas HTTP (como outros sistemas acessam)
├─ models.py         ← Modelos de dados
├─ services.py       ← Serviços (lógica principal)
└─ routes.py         ← Rotas da API
```

Cada módulo é independente — não quebra os outros módulos do sistema.

AIDD Enterprise — O Cofre de Componentes

O que é: Uma ferramenta que injeta componentes de segurança certificados no projeto.

Quando usar: Quando você precisa de componentes críticos (autenticação, criptografia) que foram verificados e "lacrado" com assinatura digital.

Analogia: É como usar peças de avião com laudo técnico — em vez de fabricar uma peça nova, você pega uma já testada e auditada.

Como usar pelo terminal:

```
# Injetar componente de autenticação JWT
python ecossistema.py enterprise inject skill auth-jwt-service
```

Como usar no chat:

```
/enterprise skill auth-jwt-service
```

O que acontece quando você roda:

1. O componente é verificado contra uma assinatura SHA-256 (uma "impressão digital" criptográfica)
 2. Se o hash não bater (alguém alterou o componente), a injeção é bloqueada
 3. Se bater, o componente é copiado para o seu projeto
-

AIDD Ops — O Gerente de Infraestrutura

O que é: Uma ferramenta que configura e sobe servidores na nuvem para rodar seu software.

Quando usar: Quando seu software está pronto e você quer colocá-lo no ar com domínio próprio, HTTPS e monitoramento.

Analogia: É o gerente de obra que liga água, luz e internet na casa que foi construída.

Como usar pelo terminal:

```
# Planejar infraestrutura
python ecossistema.py ops plan "clínica médica com agendamento"

# Testar deploy (simulação)
python ecossistema.py ops deploy staging --dry-run

# Deploy real
python ecossistema.py ops deploy production
```

Como usar no chat:

```
/ops plan "clínica médica com agendamento"
```

O que o Ops provisiona:

- **Traefik:** Proxy reverso que roteia o tráfego (como um operador de telefonia)
 - **PostgreSQL:** Banco de dados onde as informações ficam guardadas
 - **Docker:** "Caixas" isoladas onde cada parte do sistema roda
 - **HTTPS/SSL:** Certificado que garante que a comunicação é segura (cadeado no navegador)
 - **Monitoramento:** Sistema que avisa se o servidor cair (Uptime Kuma)
 - **Hardening:** Blindagem do servidor contra ataques (Ansible + devsec)
-

AIDD Bridge — O Libertador de Apps Low-Code

O que é: Uma ferramenta que tira apps de plataformas como Lovable, v0 ou Bolt e coloca para rodar em servidor próprio.

Quando usar: Quando você criou um app numa plataforma low-code e quer parar de pagar mensalidade, mantendo o app funcionando.

Analogia: É o despachante que tira seu carro do leasing e transfere pro seu nome — o carro continua o mesmo, mas agora é realmente seu.

Como usar pelo terminal:

```
# 1. Escanear o app
python ecossistema.py bridge scan ./meu-app-lovable

# 2. Converter o banco de dados
python ecossistema.py bridge convert-db ./meu-app-lovable

# 3. Empacotar para deploy
python ecossistema.py bridge pack ./meu-app-lovable --domain clinica.meudominio.c
```

Como usar no chat:

```
/bridge scan ./meu-app-lovable
```

As 6 etapas do Bridge:

Comando	O que faz
scan	Mapeia o projeto (páginas, componentes, banco)
convert-db	Converte banco Supabase para PostgreSQL puro
merge	Junta múltiplos apps num só (se necessário)
pack	Gera Docker + SSL pronto para deploy
migrate-auth	Migra contas de usuários (senhas preservadas)
destroy	Remove tudo da VPS (limpeza completa)

Os 16 Portões de Segurança (Quality Gates)

Antes de qualquer código ser considerado "pronto", ele passa por 16 verificações automáticas. São como seguranças num aeroporto — nada passa sem ser verificado.

#	Nome	O que verifica	Analogia
1	G_ECOSSISTEMA_INTEGRIDADE	Estrutura do projeto está correta	Verificar se o prédio tem todas as salas
2	G_DRIFT_NUCLEO_COMPARTILHADO	Master e Enterprise não divergiram	Verificar se dois engenheiros usaram o mesmo projeto
3	G_HARNESS_COMPAT	Todos os assistentes de IA estão sincronizados	Verificar se todos os rádios estão na mesma frequência
4	G_SEGREDOS	Não há senhas no código	Verificar se ninguém deixou a chave do cofre na rua
5	G_CLI_HELP_CONSISTENCIA	Ajuda da CLI bate com as flags reais	Verificar se o manual descreve o produto certo
6	G_COMPONENTE_AGNOSTICO	Componentes funcionam em qualquer lugar	Verificar se a peça serve em qualquer carro
7	G_ZERO_HEADLESS	Nenhum subagente invisível rodando	Verificar se não tem ninguém escondido no prédio
8	G_INFRA_COMPOSE	Docker Compose está correto	Verificar o projeto da rede elétrica
9	G_HADOLINT	Dockerfiles seguem boas práticas	Verificar se as caixas de embalagem estão corretas
10	G_TESTES_REAIS	Testes passam de verdade (sem simulação)	Testar o carro na pista, não só no desenho
11	G_HONESTIDADE_ROTULO	Não há marketing exagerado nos gates	Verificar se o relatório é honesto
12	G_ARQUITETURA_DELIVERABLE	Código respeita Clean Architecture	Verificar se a cozinha não ficou no quarto

#	Nome	O que verifica	Analogia
13	G_DEPENDENCIAS_PIN_HASH	Dependências estão amarradas com hash	Verificar se as peças são as originais
14	G_ESCRITOR_ATOMICO	Arquivos são escritos de forma atômica	Verificar se o drafts foram salvos corretamente
15	G_TRANSACTION_LOG_LRU	Log de transações não está corrompido	Verificar se o caderno de registros está íntegro
16	G_UNIVERSAL_HARNESS	Compatibilidade com todos os harnesses	Verificar se funciona em qualquer IDE

Como rodar todos os gates:

```
python ecossistema.py audit
```

Se todos passarem, o resultado é `exit 0`. Se algum falhar, é `exit 1` — e você precisa corrigir antes de prosseguir.

Como os Comandos Funcionam

Pelo Terminal (CLI)

A CLI unificada é o `ecossistema.py`. Tudo começa por ele:

```
# Ver status do ecossistema
python ecossistema.py status

# Rodar auditoria
python ecossistema.py audit

# Criar projeto
python ecossistema.py generate "minha ideia" --pasta ./destino

# Instalar governança
python ecossistema.py forge init .

# Adicionar módulo
python ecossistema.py master add-module nome

# Injetar componente
```

```
python ecossistema.py enterprise inject skill nome

# Infraestrutura
python ecossistema.py ops plan "descrição"

# Low-code
python ecossistema.py bridge scan ./app
```

Pelo Chat (Slash Commands)

Se você está usando um assistente de IA (Claude Code, Cursor, etc.), pode digitar direto no chat:

```
/generate sistema de agendamento
/forge .
/master relatorios
/enterprise skill auth-jwt
/ops plan "minha clínica"
/bridge scan ./meu-app
```

Qual a diferença? O terminal exige mais detalhes (parâmetros, caminhos). O chat é mais simples — você descreve o que quer em linguagem normal e o assistente traduz para o comando certo.

Multi-Harness: Funciona em Qualquer Assistente

O ecossistema não depende de um assistente de IA específico. Ele funciona com:

- **Claude Code** (Anthropic)
- **Cursor** (IDE com IA)
- **Antigravity** (agy)
- **MimoCode**
- **OpenCode**
- **Gemini CLI** (Google)
- **Hermes**
- **DeepSeek**

Quando você cria uma skill ou regra, ela é automaticamente copiada para a pasta de cada assistente. É como escrever uma receita uma vez e automaticamente distribuir cópias para todas as cozinhas da rede.

Como funciona na prática:

```
ecossistema-aidd/
├─ .claude/          ← Regras para Claude Code
├─ .cursor/          ← Regras para Cursor
├─ .agent/           ← Regras para AntigraVity
├─ .mimocode/        ← Regras para MimoCode
├─ .opencode/        ← Regras para OpenCode
├─ .gemini/          ← Regras para Gemini CLI
└─ .agents/          ← Regras para Hermes
```

Parte 2 — Caso Prático Completo

Objetivo

Vamos construir um **sistema de agendamento para clínica médica** do zero até produção, usando todas as ferramentas do ecossistema.

O que o sistema vai ter:

- Cadastro de médicos e horários
- Agendamento de consultas para pacientes
- Notificação por WhatsApp (futuro)
- Painel administrativo
- Acesso seguro (login)

Pré-requisitos

- Python 3.10 ou superior instalado
 - Git instalado
 - Terminal aberto
 - Um assistente de IA (Claude Code, Cursor, etc.) — opcional mas recomendado
-

Etapa 1: Criar o Projeto com o Generator

Objetivo: Transformar a ideia numa estrutura de projeto completa.

Pelo chat:

```
/generate sistema de agendamento para clínica médica com cadastro de médicos, ag
```

Pelo terminal:

```
python ecossistema.py generate "sistema de agendamento para clínica médica com ca
```

O que vai acontecer:

O Generator vai executar 8 etapas automáticas:

1. **Pesquisar** sobre o assunto do projeto
2. **Analisar** o que é necessário (cadastros, regras, etc.)
3. **Projetar** a arquitetura (que partes o sistema vai ter)
4. **Planejar** as tecnologias e estrutura
5. **Criar** os schemas e artefatos
6. **Gerar** documentação completa
7. **Auto-crítica** — verificar se tudo está correto
8. **Implementar** — *(opcional)* gerar código funcional via IA

Resultado esperado:

```
clinica-agendamento/  
├─ schemas/           ← Estruturas de dados  
├─ scripts/          ← Scripts Python  
├─ tests/             ← Testes automatizados  
├─ docs/              ← Documentação  
└─ PLANO-EXECUCAO-ESTRUTURADO.json ← Estado do projeto
```

Etapa 2: Instalar Governança com o Forge

Objetivo: Garantir que o projeto siga padrões de qualidade.

Pelo chat:

```
/forge .
```

Pelo terminal:

```
cd clinica-agendamento
python ecossistema.py forge init .
```

O que vai acontecer:

1. `AGENTS.md` será criado na raiz — regras que qualquer assistente de IA entende
2. 7 verificações automáticas (gates) serão instaladas
3. O Git será configurado para manter consistência

Etapa 3: Rodar os Gates

Objetivo: Verificar se tudo está funcionando.

```
python ecossistema.py audit
```

Resultado esperado: Todos os 16 gates passam (`exit 0`).

Se algum gate falhar, o sistema diz exatamente o que está errado e você corrige antes de continuar.

Etapa 4: Adicionar Módulos com o Master

Objetivo: Adicionar funcionalidades ao projeto.

Exemplo 1: Adicionar módulo de relatórios

```
python ecossistema.py master add-module relatorios-financeiros
```

Exemplo 2: Adicionar módulo de prontuário eletrônico

```
python ecossistema.py master add-module prontuario-eletronico
```

Cada módulo criado tem:

- `domain/` — Regras de negócio (ex: "agendamento só pode ser em horário comercial")
 - `application/` — Casos de uso (ex: "criar agendamento", "cancelar agendamento")
 - `infrastructure/` — Banco de dados (ex: tabelas de médicos e consultas)
 - `interfaces/` — API (ex: endpoints REST para o frontend consumir)
-

Etapa 5: Injetar Autenticação com o Enterprise

Objetivo: Proteger o sistema com login seguro.

```
python ecossistema.py enterprise inject skill auth-jwt-service
```

O que isso faz:

- Injeta um componente de autenticação JWT (um padrão seguro de login)
 - O componente foi testado e tem assinatura SHA-256 (impressão digital criptográfica)
 - Se alguém tentar adulterar o componente, a injeção é bloqueada
-

Etapa 6: Configurar Infraestrutura com o Ops

Objetivo: Preparar o servidor para rodar o sistema em produção.

Planejar:

```
python ecossistema.py ops plan "clínica médica com agendamento de consultas, cada
```

Testar (simulação):

```
python ecossistema.py ops deploy staging --dry-run
```

Deploy real:

```
python ecossistema.py ops deploy production
```

O que o Ops vai provisionar:

- Um servidor (VPS) na nuvem
 - Docker com cada parte do sistema num container isolado
 - Traefik para roteamento (apontar domínio para o sistema certo)
 - PostgreSQL para o banco de dados
 - HTTPS automático (cadeado no navegador)
 - Uptime Kuma para monitoramento (aviso se o servidor cair)
 - Hardening Ansible (blindagem contra ataques)
-

Etapa 7 (Alternativa): Se o App Veio do Lovable

Se em vez de criar do zero, você já tem um app no Lovable:

Libertar o app:

```
# 1. Escanear
python ecossistema.py bridge scan ./app-lovable

# 2. Converter banco (Supabase → PostgreSQL)
python ecossistema.py bridge convert-db ./app-lovable

# 3. Migrar contas (opcional)
python ecossistema.py bridge migrate-auth \
  --source postgres://supabase-host:5432/postgres \
  --target postgres://minha-vps:5432/postgres

# 4. Empacotar com Docker + SSL
python ecossistema.py bridge pack ./app-lovable --domain clinica.meudominio.com
```

Depois, seguir da Etapa 2 (Forge) em diante.

Fluxo Resumido

```
IDEIA
|
▼
/generate "minha ideia"      ← Cria projeto completo
|
▼
/forge .                    ← Instala regras de qualidade
|
▼
python ecossistema.py audit ← Verifica tudo
|
▼
/master <modulo>            ← Adiciona funcionalidades
|
▼
/enterprise auth jwt        ← Protege com login seguro
|
```

▼
/ops plan "descrição" ← Sobe na nuvem
|
▼
PRODUÇÃO (app rodando com HTTPS, monitoramento e segurança)

Parte 3 — Referência Rápida

Todos os Comandos

Comando	O que faz	Exemplo
<code>status</code>	Mostra estado do ecossistema	<code>python ecossistema.py status</code>
<code>audit</code>	Roda os 16 gates	<code>python ecossistema.py audit</code>
<code>forge init</code>	Instala governança	<code>python ecossistema.py forge init .</code>
<code>forge inject</code>	Injeta componente	<code>python ecossistema.py forge inject skill x</code>
<code>generate</code>	Cria projeto	<code>python ecossistema.py generate "ideia" --pasta ./x</code>
<code>master add-module</code>	Adiciona módulo	<code>python ecossistema.py master add-module x</code>
<code>enterprise inject</code>	Injeta componente certificado	<code>python ecossistema.py enterprise inject skill x</code>
<code>ops plan</code>	Planeja infraestrutura	<code>python ecossistema.py ops plan "descrição"</code>
<code>ops deploy</code>	Faz deploy	<code>python ecossistema.py ops deploy staging --dry-run</code>
<code>bridge scan</code>	Escaneia app low-code	<code>python ecossistema.py bridge scan ./app</code>
<code>bridge convert-db</code>	Converte banco	<code>python ecossistema.py bridge convert-db ./app</code>
<code>bridge pack</code>	Empacota Docker	<code>python ecossistema.py bridge pack ./app --domain x</code>

Comando	O que faz	Exemplo
<code>bridge merge</code>	Junta apps	<code>python ecossistema.py bridge merge ./a ./b --output ./c</code>
<code>bridge migrate-auth</code>	Migra contas	<code>python ecossistema.py bridge migrate-auth --source x --target y</code>
<code>bridge destroy</code>	Remove da VPS	<code>python ecossistema.py bridge destroy nome --domain x</code>
<code>components sync</code>	Sincroniza componentes	<code>python ecossistema.py components sync --tipo todos</code>
<code>dependencia bootstrap</code>	Instala dependências	<code>python ecossistema.py dependencia bootstrap</code>
<code>orchestrate</code>	Orquestra multi-frente	<code>python ecossistema.py orchestrate ./plano</code>

Glossário

Termo	Significado
CLI	Interface de linha de comando — usar o terminal em vez de botões
Slash Command	Comando no chat do assistente de IA (começa com <code>/</code>)
Quality Gate	Verificação automática que bloqueia código não-conforme
Vendor Lock-in	Ficar preso a uma plataforma difícil de sair
Clean Architecture	Organizar código em camadas (regras, dados, interface)
Fatia Vertical	Módulo completo e independente do sistema
Mock	Simulação falsa (proibido no ecossistema)
Result Monad	Padrão de "sucesso ou falha" em vez de quebrar o programa
SHA-256	Impressão digital criptográfica para verificar integridade
Zero-Trust	Não confiar em nada sem verificar primeiro
WAL	Write-Ahead Logging — modo seguro de escrever no banco SQLite
PostgreSQL	Banco de dados robusto e gratuito
Docker	Tecnologia de "containers" — isolar cada parte do sistema

Termo	Significado
HTTPS	Comunicação segura (o cadeado no navegador)
Traefik	Proxy reverso — roteia o tráfego para o serviço certo
VPS	Virtual Private Server — servidor seu na nuvem
Harness	Assistente de IA (Claude, Cursor, etc.)
Subagente	IA que trabalha em segundo plano numa tarefa específica
Context-Purge	Limpar o contexto da IA entre etapas (economizar tokens)
Tokens	Unidade de medida de texto para IAs (1 token ≈ 4 caracteres)

Solução de Problemas

Problema	Causa	Solução
ModuleNotFoundError	Dependências não instaladas	<code>python ecossistema.py --auto-bootstrap status</code>
<code>exit 1</code> no audit	Gates falhando	Ler a mensagem de erro e corrigir o que o gate aponta
Bridge não encontra SQL	Estrutura diferente	Verificar se o projeto tem <code>supabase/migrations/</code>
Master cria imports cruzados	Módulos acoplados	Comunicar via EventBus, nunca import direto
Ops precisa de Linux	Ansible não roda no Windows	Usar WSL2 ou um container Docker
Tokens esgotando rápido	Contexto grande demais	Usar context-purge (o ecossistema faz isso automaticamente)

Estrutura do Repositório

ecossistema-aidd/	
├─ AGENTS.md	← Regras fundamentais
├─ ecossistema.py	← CLI unificada
├─ gates/	← 16 Quality Gates
├─ componentes/	← Código compartilhado

		compartilhado/	← Núcleo (Database, Result, EventBus)
		aidd-master/	← Componentes do Master
		tools/	← As 6 ferramentas
		aidd-forge/	← Governança
		aidd-generator/	← Fábrica
		aidd-master/	← Módulos
		aidd-enterprise/	← Segurança
		aidd-ops/	← Infraestrutura
		aidd-bridge/	← Low-code → VPS
		docs/	← Documentação
		MANUAL-COMPLETO.md	← Este manual
		ARQUITETURA-ECOSSISTEMA.md	
		CLI-REFERENCIA.md	
		MIGRACAO-PROJETO-EXISTENTE.md	
		CONTRIBUICAO.md	
		planos/	← Planos táticos e de auditoria
		protocolos/	← Protocolos canônicos
		relatorios/	← Relatórios formais
		prompts/	← Biblioteca de prompts
		features/	← Documentação de features
		explicacoes/	← Explicações detalhadas
		melhorias/	← Registro de melhorias
		testes/	← Baterias de testes E2E

*Manual gerado em Setembro de 2026 pelo Ecossistema AIDD.
Distribuído sob a licença MIT.*